

WEB GELİŞTİRME

FastAPI, modern, hızlı (yüksek performanslı) ve Python 3.6+ ile web API'leri oluşturmak için tasarlanmış bir web **çerçevesidir** (**framework**). Otomatik API dokümantasyonu, **tür ipuçları** (**type hints**) desteği ve yüksek performansı ile popülerdir.

FastAPI'nin Özellikleri;

- Hızlı: NodeJS ve Go kadar hızlı performans
- Kolay: Öğrenmesi ve kullanması kolay
- Otomatik Dokümantasyon: Swagger UI ve ReDoc otomatik oluşturulur
- Type Hints: Python type hints ile güçlü tip kontrolü
- Asenkron Destek: async/await tam desteği
- Veri Doğrulama (validation): Pydantic ile otomatik veri doğrulama
- Dependency Injection: Bağımlılık enjeksiyonu sistemi

Kurulum için aşağıdaki iki komut çalıştırılır;

```
$ pip install fastapi
$ pip install "uvicorn[standard]" # ASGI sunucusu
```

İlk FastAPI Uygulaması

İlk uygulamamız aşağıdaki gibi olsun;

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def ana_sayfa():
    return {"mesaj": "Merhaba FastAPI!"}

@app.get("/selam/{isim}")
def selam_ver(isim: str):
    return {"mesaj": f"Merhaba {isim}!"}
```

Uygulamayı çalıştırma:

```
$ uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['D:\\Python\\Samples\\fastapi']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [13032] using WatchFiles
INFO: Started server process [3376]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Tarayıcıda aşağıdaki URL'ler açılır;

- <http://127.0.0.1:8000> - Ana sayfa
- <http://127.0.0.1:8000/selam/Alı> - Selamlaşma
- <http://127.0.0.1:8000/docs> - Otomatik API dokümantasyonu (Swagger UI)
- <http://127.0.0.1:8000/redoc> - Alternatif dokümantasyon (ReDoc)

Path Parametreleri

URL'den değer almak için path parametreleri kullanılır;

```
def kullanıcı_getir(kullanıcı_id: int):
    return {"kullanıcı_id": kullanıcı_id, "isim": "Ali"}
```

```
@app.get("/urunler/{kategori}/{urun_id}")
def urun_detay(kategori: str, urun_id: int):
    return {
        "kategori": kategori,
        "urun_id": urun_id,
        "isim": f"{kategori} Ürünü {urun_id}"
    }
```

Query Parametreleri

URL'deki query string'den değerler aşağıdaki şekilde alınabilir;

```
@app.get("/ara")
def ara(q: str, limit: int = 10, offset: int = 0):
    """
    q: arama terimi (zorunlu)
    limit: sonuç sayısı (varsayılan 10)
    offset: başlangıç noktası (varsayılan 0)
    """
    return {
        "arama": q,
        "limit": limit,
        "offset": offset,
        "sonuclar": [f"Sonuç {i}" for i in range(offset, offset + limit)]
    }

@app.get("/filtrele")
def filtrele(kategori: Optional[str] = None, min_fiyat: Optional[float] = None):
    """Opsiyonel parametreler"""
    filtre = {}
    if kategori:
        filtre["kategori"] = kategori
    if min_fiyat:
        filtre["min_fiyat"] = min_fiyat
    return {"filtre": filtre}
```

Buradaki uygulama aşağıdaki örneklerde verilen query string'den değerler alabilir;

- <http://127.0.0.1:8000/ara?q=python>
- <http://127.0.0.1:8000/ara?q=python&limit=5&offset=10>
- http://127.0.0.1:8000/filtrele?kategori=kitap&min_fiyat=50

Request Body

POST isteklerinde veri göndermek için Pydantic modelleri kullanılır:

```
from typing import Optional

class Kullanici(BaseModel):
    isim: str = Field(..., min_length=2, max_length=50)
    email: EmailStr
    yas: int = Field(..., gt=0, lt=150)
    aktif: bool = True
    aciklama: Optional[str] = None

@app.post("/kullanici")
def kullanici_olustur(kullanici: Kullanici):
    return {
        "mesaj": "Kullanıcı oluşturuldu",
        "kullanici": kullanici
    }

class UrunGuncelle(BaseModel):
```

```
    isim: Optional[str] = None
    fiyat: Optional[float] = Field(None, gt=0)
    stok: Optional[int] = Field(None, ge=0)

@app.put("/urun/{urun_id}")
def urun_guncelle(urun_id: int, urun: UrunGuncelle):
    return {
        "urun_id": urun_id,
        "guncellenen": urun.dict(exclude_unset=True) # Sadece gönderilen alanlar
    }
```

HTTP Yöntemleri

Beş adet http yöntemine ilişkin örnekler aşağıda verilmiştir;

```
# GET - Veri okuma
@app.get("/urunler")
def urunleri_listele():
    return [{"id": 1, "isim": "Ürün 1"}, {"id": 2, "isim": "Ürün 2"}]

# POST - Yeni veri oluşturma
@app.post("/urunler", status_code=status.HTTP_201_CREATED)
def urun_olustur(urun: dict):
    return {"mesaj": "Ürün oluşturuldu", "urun": urun}

# PUT - Veri güncelleme (tüm alanlar)
@app.put("/urunler/{urun_id}")
def urun_guncelle_put(urun_id: int, urun: dict):
    return {"mesaj": f"Ürün {urun_id} güncellendi", "urun": urun}

# PATCH - Kısmi güncelleme
@app.patch("/urunler/{urun_id}")
def urun_guncelle_patch(urun_id: int, urun: dict):
    return {"mesaj": f"Ürün {urun_id} kısmen güncellendi", "urun": urun}

# DELETE - Veri silme
@app.delete("/urunler/{urun_id}", status_code=status.HTTP_204_NO_CONTENT)
def urun_sil(urun_id: int):
    return None # 204 için içerik dönmeyin
```

Response Models

Dönen veriyi şekillendirmek için `response_model` kullanılır:

```
#     email: EmailStr
#     sifre: str

class KullaniciCikis(BaseModel):
    id: int
    isim: str
    email: EmailStr
    # sifre alanı yok!

@app.post("/giris", response_model=KullaniciCikis)
def giris_yap(kullanici: KullaniciGiris):
    # Veritabanından kullanıcı bul ve doğrula
    return {
        "id": 1,
        "isim": "Ali Veli",
        "email": kullanici.email,
        "sifre": "hash123" # Bu alan response'a dahil edilmez
    }
```

Hata Yönetimi

```
# Basit hata
@app.get("/kullanici/{kullanici_id}")
def kullanici_getir(kullanici_id: int):
    if kullanici_id > 100:
        raise HTTPException(status_code=404, detail="Kullanıcı bulunamadı")
    return {"kullanici_id": kullanici_id}

# Detaylı hata
@app.get("/urun/{urun_id}")
def urun_getir(urun_id: int):
    if urun_id < 1:
        raise HTTPException(
            status_code=400,
            detail="Ürün ID 1'den küçük olamaz",
            headers={"X-Error": "Geçersiz ID"}
        )
    return {"urun_id": urun_id}
```

Dependency Injection

Ortak kodları paylaşmak ve bağımlılıkları enjekte etmek için:

```
# Basit dependency
def ortak_parametreler(q: Optional[str] = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items/")
def itemleri_oku(common: dict = Depends(ortak_parametreler)):
    return common

# Token kontrolü
def token_kontrol(x_token: str = Header(...)):
    if x_token != "gizli-token":
        raise HTTPException(status_code=400, detail="Geçersiz token")
    return x_token

@app.get("/korunmus")
def korunmus_endpoint(token: str = Depends(token_kontrol)):
    return {"mesaj": "Erişim izni verildi"}

# Veritabanı bağlantısı
def veritabanı_baglantisi():
    db = {"baglanti": "aktif"}
    try:
        yield db
    finally:
        print("Veritabanı bağlantısı kapatıldı")

@app.get("/veriler")
def veri_oku(db: dict = Depends(veritabanı_baglantisi)):
    return {"veritabanı": db["baglanti"], "veriler": [1, 2, 3]}
```

Asenkron Endpoint'ler

```
import aiohttp

@app.get("/sync")
def senkron_endpoint():
    # Normal fonksiyon
```

```

    return {"tip": "senkron"}

@app.get("/async")
async def asenkron_endpoint():
    # Asenkron fonksiyon
    await asyncio.sleep(1)
    return {"tip": "asenkron"}

@app.get("/harici-api")
async def harici_api_cagir():
    async with aiohttp.ClientSession() as session:
        async with session.get("https://api.github.com") as response:
            veri = await response.json()
            return {"durum": response.status, "veri": veri}

```

Dosya Yükleme

Dosya yükleme (file upload) örneği aşağıda verilmiştir;

```

from typing import List

@app.post("/dosya-yukle")
async def dosya_yukle(dosya: UploadFile = File(...)):
    icerik = await dosya.read()
    return {
        "dosya_adi": dosya.filename,
        "boyut": len(icerik),
        "icerik_tipi": dosya.content_type
    }

@app.post("/coklu-dosya")
async def coklu_dosya_yukle(dosyalar: List[UploadFile] = File(...)):
    return [
        {
            "dosya_adi": dosya.filename,
            "boyut": len(await dosya.read())
        }
        for dosya in dosyalar
    ]

```

Çapraz Kaynak Paylaşımı

Çapraz kaynak paylaşımı (Cross-Origin Resource Sharing-CORS) tekniğinde **önyüz** (frontend) uygulamalarından API'ye erişim kontrol altına alınır;

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "https://myapp.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Geliştirme için tüm kaynaklara izin ver (production'da kullanma!)
allow_origins=["*"]

```

Arka Planda Görev Çalıştırma

İstek tamamlandıktan sonra **arka planda görev çalıştırma** (background task):

```

def email_gonder(email: str, mesaj: str):
    # Email gönderme simülasyonu

```

```
import time
time.sleep(5) # Uzun süren işlem
print(f"{email} adresine email gönderildi: {mesaj}")

@app.post("/kayit")
async def kayit_ol(email: EmailStr, arka_plan: BackgroundTasks):
    arka_plan.add_task(email_gonder, email, "Hoş geldiniz!")
    return {"mesaj": "Kayıt başarılı, email gönderiliyor..."}
```

Veritabanı Entegrasyonu (SQLAlchemy)

```
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session

# Veritabanı kurulumu
SQLALCHEMY_DATABASE_URL = "sqlite:///./test.db"
engine = create_engine(SQLALCHEMY_DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

# Model tanımı
class KullaniciDB(Base):
    __tablename__ = "kullanici"
    id = Column(Integer, primary_key=True, index=True)
    isim = Column(String)
    email = Column(String, unique=True, index=True)

Base.metadata.create_all(bind=engine)

# Dependency
def veritabani_oturumu():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# CRUD (Create-Read-Update-Delete) işlemleri
@app.post("/kullanici/")
def kullanici_olustur(kullanici: Kullanici, db: Session = Depends(veritabani_oturumu)):
    db_kullanici = KullaniciDB(isim=kullanici.isim, email=kullanici.email)
    db.add(db_kullanici)
    db.commit()
    db.refresh(db_kullanici)
    return db_kullanici

@app.get("/kullanici/")
def kullanicilari_listele(skip: int = 0, limit: int = 10, db: Session = Depends(veritabani_oturumu)):
    kullanicilar = db.query(KullaniciDB).offset(skip).limit(limit).all()
    return kullanicilar

# Authentication - JWT Token
from jose import JWTError, jwt
from passlib.context import CryptContext

SECRET_KEY = "gizli-anahtar-buraya"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
```

```
def sifre_hash(sifre: str) -> str:
    return pwd_context.hash(sifre)

def sifre_dogrula(sifre: str, hash_sifre: str) -> bool:
    return pwd_context.verify(sifre, hash_sifre)

def token_olustur(veri: dict):
    to_encode = veri.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

@app.post("/token")
def giris(email: str, sifre: str):
    # Kullanıcı doğrulama (basitleştirilmiş)
    if email == "test@test.com" and sifre == "sifre123":
        token = token_olustur({"sub": email})
        return {"access_token": token, "token_type": "bearer"}
    raise HTTPException(status_code=401, detail="Geçersiz kimlik bilgileri")
```

Modüler Yapı

Büyük projelerde kodları modüllere ayırmak için `APIRouter` yapısı kullanılır:

```
# routers/kullaniciilar.py
from fastapi import APIRouter

router = APIRouter(
    prefix="/kullaniciilar",
    tags=["kullaniciilar"]
)

@router.get("/")
def kullaniciilari_listele():
    return [{"id": 1, "isim": "Ali"}]

@router.get("/{kullanici_id}")
def kullanici_detay(kullanici_id: int):
    return {"id": kullanici_id, "isim": "Ali"}
```

```
# main.py
from fastapi import FastAPI
from routers import kullaniciilar

app = FastAPI()

app.include_router(kullaniciilar.router)
```

Gerçek Zamanlı İletişim

Gerçek zamanlı iletişim için `WebSocket` yapısı kullanılır:

```
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            veri = await websocket.receive_text()
            await websocket.send_text(f"Mesaj alındı: {veri}")
    except:
```

```
print("Bağlantı kapatıldı")
```

API'leri Test Etme

İlk yöntem **tarayıcı** (**browser**) üzerinden test etmektir.

```
http://localhost:8000/docs
```

İkinci yöntem CURL dili ile test etmektir;

```
curl -X GET "http://localhost:8000/items/5"
```

Üçüncü yöntem ise Python dili ile **request** ile test etmektir;

```
import requests
response = requests.get("http://localhost:8000/items/5")
print(response.json())
```

En İyi Uygulamalar

- Type hints her yerde kullanın
- Response models ile veri şekillendirme yapın
- Dependency injection ile kod tekrarını önleyin
- Async endpoint'leri I/O işlemleri için kullanın
- Router'larla kodları modüllere ayırın
- Environment variables ile yapılandırma yapın
- Logging ekleyin
- Rate limiting uygulayın
- HTTPS kullanın (production'da)
- Input validation'ı Pydantic'e bırakın
- Database connection pooling kullanın
- Background tasks ile uzun işlemleri yönetin

Web Proje Yapısı

```
my-fastapi-app/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── kullanıcı.py
│   ├── routers/
│   │   ├── __init__.py
│   │   ├── kullanıcılar.py
│   │   └── ürünler.py
│   ├── schemas/
│   │   ├── __init__.py
│   │   └── kullanıcı.py
│   ├── database.py
│   └── dependencies.py
├── tests/
│   ├── __init__.py
│   └── test_main.py
├── requirements.txt
└── .env
```